

Функции и переменные

Основы языка C, лекция 2

Повторяем: переменная

- Переменная — именованная область памяти

- `int x;`

`int y;`

`x = 2;`

`y = 5;`



- `int x, y;` // через запятую

`x = 2;`

`y = 5;`

- `int x = 2, y = 5;` // явная инициализация

Повторяем: сложение чисел

- `#include <stdio.h>`

```
int main ( ) {
```

```
    int x, y, res;           // декларация переменных
```

```
    scanf("%d", &x);         // ввод данных
```

```
    scanf("%d", &y);
```

```
                                // обработка данных
```

```
    res = x + y;
```

```
                                // вывод результатов
```

```
    printf("%d plus %d is %d\n", x, y, res);
```

```
    return 0;
```

```
}
```

Повторяем: типы данных

- Целочисленные типы данных
 - char %hhd
 - short %hd
 - int %d
 - long %ld
 - long long %lld
- unsigned, *signed* %d → %u
- данные с плавающей точкой
 - float %f %g %e
 - double %lf

Что такое функция

- Математика: отображение множества входных аргументов на множество результатов.

$$y = \sin(x) \quad y = x^k$$

- Программирование: именованный кусок кода. Фрагмент кода, к которому можно *обратиться (call)* из другого места программы.

`scanf`, `printf` — уже вызывали стандартные функции

- У каждой функции есть идентификатор — имя функции.

Зачем нужны функции

- Модульность кода — **разбиение 1 задачи на много маленьких задач** — наша цель
 - повторное использование кода
 - уже написано
 - меньше писать
 - легче отлаживать
 - скрывает несущественные для других частей детали реализации (printf, scanf, qsort)
 - можно переписать 1 функцию
 - взять из библиотеки / написать самому

Математические функции

- `#include <stdio.h>` `// ex_sin.c`
`#include <math.h>`
`int main( ) {`
 `double x, y;`
 `x = M_PI;`
 `y = sin(x) + 2*cos(x);`
 `printf("%lf\n", y);`
 `return 0;`
`}`
- `gcc -Wall -Wextra -lm -o ex_sin ex_sin.c`
Указать, что нужно линковать библиотеку стандартных математических функций

man 3 sin

```
tatyderb@nearbird: /mnt/c/work/conline/1_int/text
SIN(3) Linux Programmer's Manual SIN(3)
NAME
    sin, sinf, sinl - sine function
SYNOPSIS
    #include <math.h>

    double sin(double x);
    float sinf(float x);
    long double sinl(long double x);

    Link with lm.

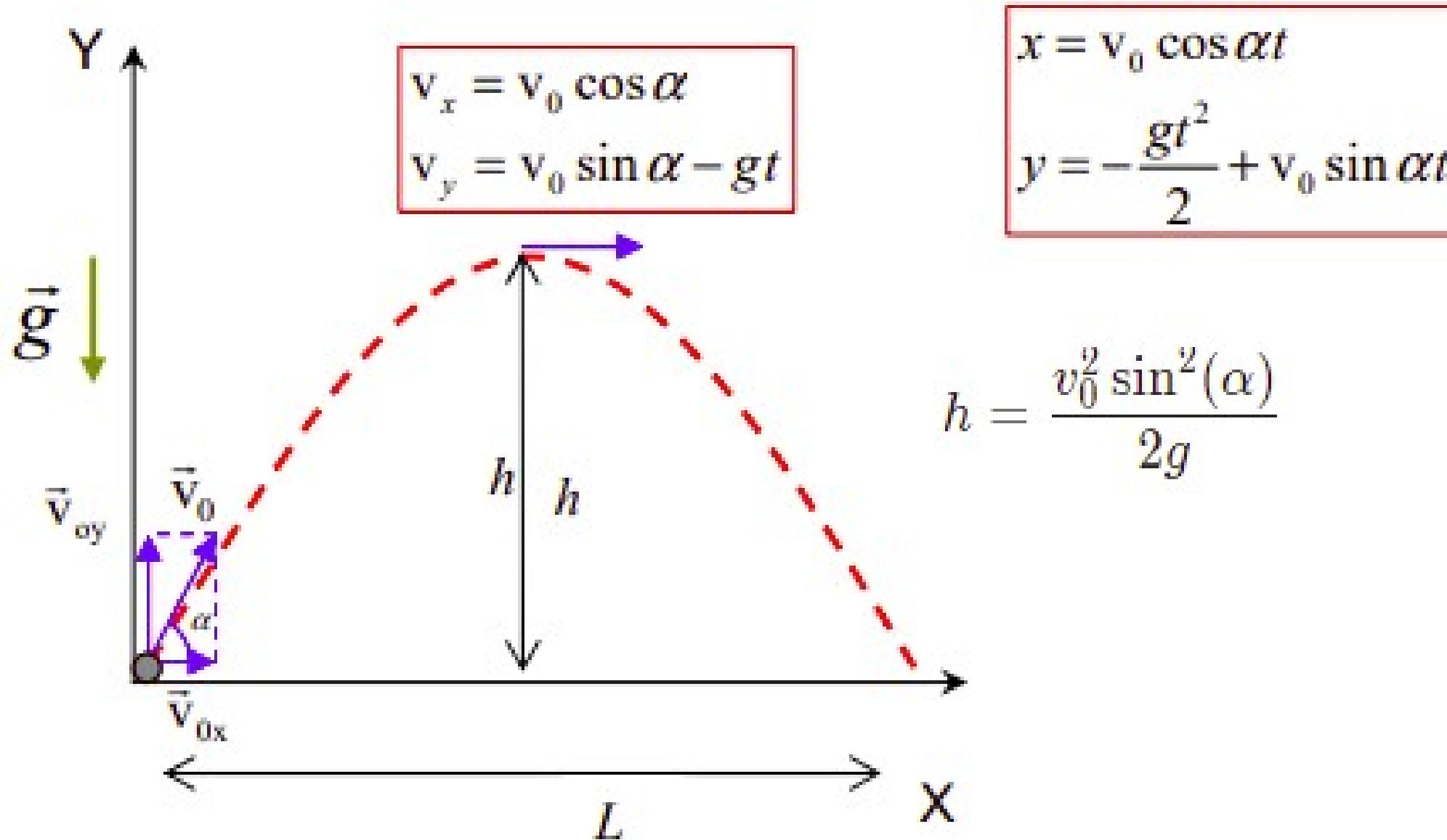
    Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

    sinf(), sinl():
        _ISOC99_SOURCE || _POSIX_C_SOURCE >= 200112L
        || /* Since glibc 2.19: */ _DEFAULT_SOURCE
        || /* Glibc versions <= 2.19: */ _BSD_SOURCE || _SVID_SOURCE
DESCRIPTION
    These functions return the sine of x, where x is given in radians.
RETURN VALUE
    On success, these functions return the sine of x.

    If x is a NaN, a NaN is returned.

Manual page sin(3) line 1/65 33% (press h for help or q to quit)
```

Тело, брошенное под углом к горизонту



Создадим свою функцию foo

- `#include <stdio.h>`

`//  $x^2 + 2x + 3$  — реализовали функцию foo:`

```
float foo (float x) {  
    float res;  
    res =  $x^2 + 2x + 3$ ;  
    return res;  
}
```

```
return  $x^2 + 2x + 3$ ;
```

```
int main ( ) {  
    float y1, y2, y3;  
    y1 = foo (0);    // call foo  
    y2 = foo (1);    // call foo  
    printf("%f %f\n", y1, y2);  
}
```

Создадим свою функцию foo

- #include <stdio.h>

// $ax^2 + bx + c$ — реализовали функцию foo:

```
float foo (float a, float b, float c, float x) {  
    float res;  
    res = a*x*x + b*x + c;    return a*x*x + b*x + c;  
    return res;  
}
```

```
int main ( ) {  
    float y1, y2, y3;  
    y1 = foo (1, 2, 3, 0.5);           // call foo  
    y2 = foo (-3.1, 5.16, -0.01, 10); // call foo  
    printf("%f %f\n", y1, y2);  
}
```

`y = foo(1, 2, 31, 4.5);`

- Создаются переменные `a`, `b`, `c`, `x` и в них записываются значения `1`, `2`, `31`, `4.5`
 - неявное преобразование типа
- управление передается в функцию
выполняются одна за другой инструкции
`float res;`
`res = a*x*x + b*x + c;`
- когда выполнение доходит до **`return res;`**
выход из функции в точку вызова
- Значение выражения `foo(1, 2, 31, 4.5)` — что вернул оператор `return`

Создание своей функции

- тип имя_функции (список аргументов) {
 тело функции
}
- **имя_функции** — идентификатор (как придумать?)
- **тип** — тип возвращаемого значения **void**
- **список аргументов**: через запятую или пустой
 - для каждого аргумента — свой тип и имя (придумать самим)
- **return** — возвратиться из функции **return ;**

void — нечего возвращать

- #include <stdio.h>

```
void hi ( ) {  
    printf("Здравствуй, уважаемый \n");  
    return ;           // это можно не писать  
}  
  
int main ( ) {        // main — тоже функция  
    hi ( );  
    hi ( );  
    hi ( );  
    hi ( );  
    hi ( );  
    return 0;         // кто вызвал main и кому вернули 0?  
}
```

Безграмотность

- **void** main () { ... }
 - по такой книжке нельзя учиться, она плохая
 - это не язык C
- exit code
 - проверяющая система (0 — это OK)
 - другие программы в комплексе, например обработка изображений

аргумент group

- #include <stdio.h>

```
void hi ( int group ) {  
    printf("Здравствуйте, группа %d!\n", group);  
  
}  
  
int main ( ) {  
    hi ( 951 ); // Здравствуйте, группа 951!  
    hi ( 954 ); // Здравствуйте, группа 954!  
    hi ( 978 ); // Здравствуйте, группа 978!  
    hi ( 916 ); // Здравствуйте, группа 916!  
    hi ( );      // ошибка, мало аргументов  
    return 0;  
}
```

Запрещено

- Функции с одинаковыми именами
- Значения по умолчанию
- *Переменное число аргументов — позже printf*
- Реализация функции внутри другой функции (pascal, python, ...)

```
int main ( ) {  
    void hi ( ) {  
        printf("Здравствуй");  
    }  
    // дальше идет функция main  
    hi ( );  
}
```

ИМЕНА И ТИПЫ

- Имя функции должно быть значимым
- 1 функция — 1 действие
 - get_font, set_size, draw_line, to_upper
 - is_empty, ~~is_not_empty~~
- Если не можете назвать функцию, разбейте ее на несколько мелких функций
- тип — у каждого аргумента
long power(**int** x, **int** n) // ok
long power(**int** x, n) // error

Code style

- `/*` описание что делает функция `*/`
`int main ( )`
`{`
 тело функции с отступом
`}`
- Скобки на 1 позиции,
на этой строке ничего больше не пишем
- отступ в теле функции
- после `,` в аргументах ставим пробел
- между функциями оставьте пустую строку

Нет дара предвидения

- ДО вызова функции нужна информация компилятору о:
 - количестве аргументов и типе каждого аргумента
 - типе возвращаемого значения
- Сначала вас учат,
потом вы используете знания

Какую функцию раньше?

-

```
void foo ( ) {  
    ...  
    bzz( );           // что такое bzz ???  
    ...  
}
```

```
void bzz ( ) {  
    ...  
    foo ( );  
    ...  
}
```

Прототип функции

- Информация компилятору об:
 - имени функции,
 - возвращаемом типе,
 - количестве аргументов и их типах
- Вместо **{ тело функции }** пишем **;**
- ```
void bzz(); // прототип функции bzz (prototype)
void foo() {
 bzz(); // вызов bzz (call)
}
void bzz() { // реализация bzz (implementation)
 foo();
}
```

# Прототип и реализация

- // Прототип  
тип имя\_функции (список *типов* аргументов) ;
- // Реализация  
тип имя\_функции (список аргументов) {  
    тело функции  
}
- зачем писать имена аргументов? Понятность  
  
    long power (int x, int n);     // x в степени n  
  
    long power (int, int);         // где степень?
  - понятно компилятору, человеку НЕТ

# Из Цельсия в Фаренгейты

- $F = C * 9 / 5 + 32$

|   |    |     |     |      |
|---|----|-----|-----|------|
| C | 0  | 50  | -40 | 1    |
| F | 32 | 122 | -40 | 33.8 |

- Отправлять только реализацию функции
  - main уже в проверяющей системе
  - обязательно писать самим main и тестировать функцию

# Тестирование функций

- `#include <assert.h>`
- `assert (fahr(0) == 32); // целые числа`  
`assert (fahr(-40) == -40);`

надо два ==

- сравниваем дробные числа  
 $|a - b| < \text{eps}$

```
#include <math.h>
```

```
#define EPS 0.0001
```

```
assert (fabs(fahr(1) - 33.8) < EPS);
```

# Характеристики переменной

- **Имя**

- **Тип**



- **Значение**

- **Область видимости**

в каком месте программы можно обратиться к данной переменной

- **Время жизни**

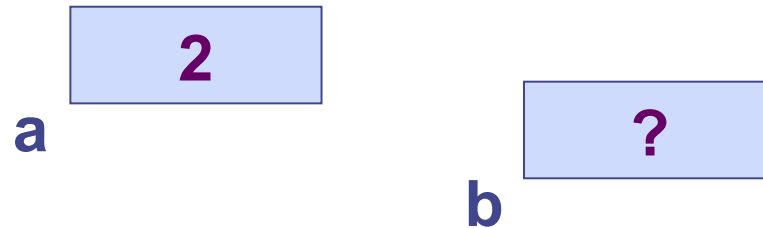
Когда переменная создается и когда уничтожается

- `int x; // чему равно x сейчас?`

# Локальные переменные

- область видимости — от места декларации до конца соответствующего блока { ... } (if, циклы)
- время жизни — до конца выполнения этого блока
- начальное значение — "мусор" (что было записано в этом месте памяти, то и осталось)

```
• int main () {
 int a = 2,
 b;
 printf ("%d %d\n", a, b); // 2 ???
 return 0;
}
```



# Аргументы функций

- Область видимости — эта функция
- Время жизни — время 1 (одного) вызова этой функции
  - создаются при вызове функции
  - уничтожаются после выхода из нее
  - и так для каждого вызова — новые наборы аргументов
- Начальное значение — значение аргумента при вызове

# Аргументы функций

- `int sum (int x, int y) {`

`return x + y;`

`}`

`x`

2

`y`

3

`int main ( ) {`

`int a = 2, b = 3;`

`printf("%d %d %d\n", a, b, sum(a, b) ); // 2 3 5`

`a = sum(a, b);`

`printf("%d %d\n", a, b);`

`// ? ?`

2

`b = sum(a, b);`

`a`

`printf("%d %d\n", a, b);`

`// ? ?`

3

`return 0;`

`b`

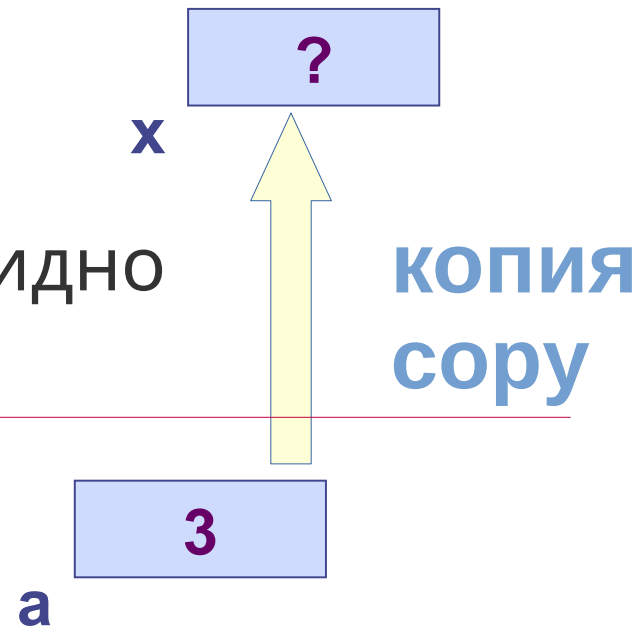
`}`

# Передача аргументов

- Аргументы передаются только по значению (копии)

- ```
void inc (int x) {  
    x = x + 1;    // x++;  
    // a = a + 1; // нельзя, не видно  
}
```

```
int main ( ) {  
    int a = 3;  
    inc (a);  
    inc (a);  
    printf("%d\n", a);    // ??  
}
```



Когда копий не хватает?

- изменить передаваемое значение
(два передаваемых значения)

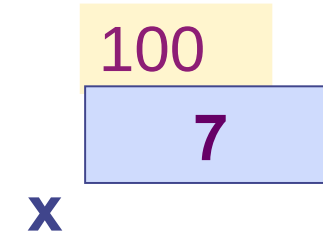
```
void swap (int x, int y) { ... }  
  
int main( ) {  
    int a = 2, b = 7;  
    swap(a, b);          // a = 7; b = 2;  
}
```
- вернуть несколько значений
из минут с 0:00 получить часы и минуты

```
h, m = mm2hm(135); // в C так нельзя!
```
- Что делать?

```
scanf("%d", &x);
```

Адрес переменной

- Пусть переменная x (размером в несколько байт) начинается с байта с номером 100 и там записано число 7.



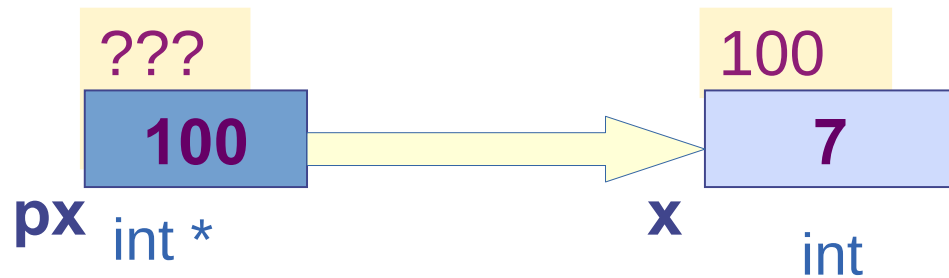
- **Адрес** — это номер байта (100, число)
- Переменная может быть расположена в нескольких байтах подряд.

Адрес переменной — номер ее первого байта.

- Чтобы узнать по адресу переменной, какое число в ней хранится, нужно еще знать ее тип:
 - размер переменной
 - как интерпретировать (int, float, unsigned)

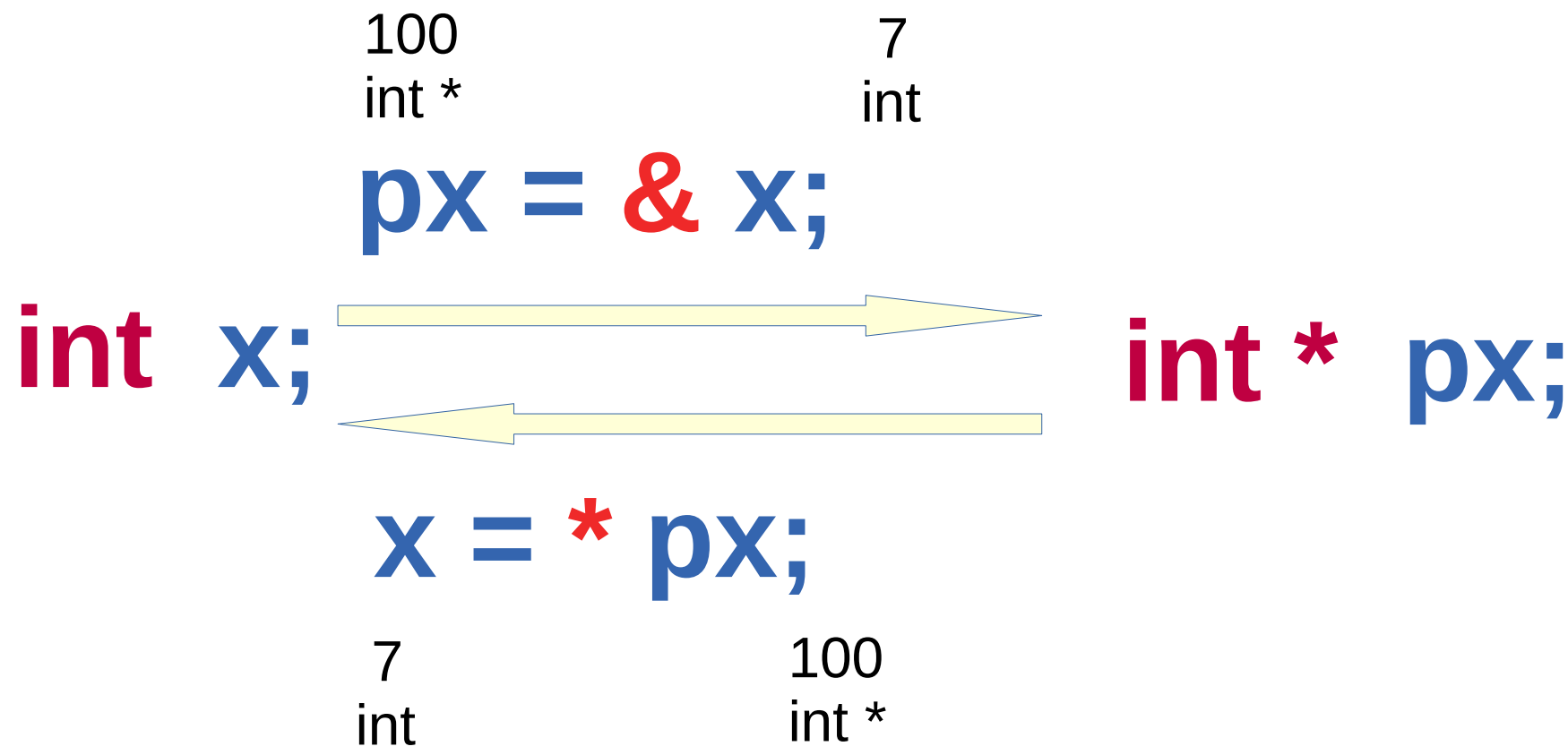
Тип "указатель на ..."

- `2 * 3` // умножение
- `int x = 7;`
`int * px;` // `px` типа указатель на `int`
// pointer to `int`
`px = &x;` // `&` - вычислить адрес переменной

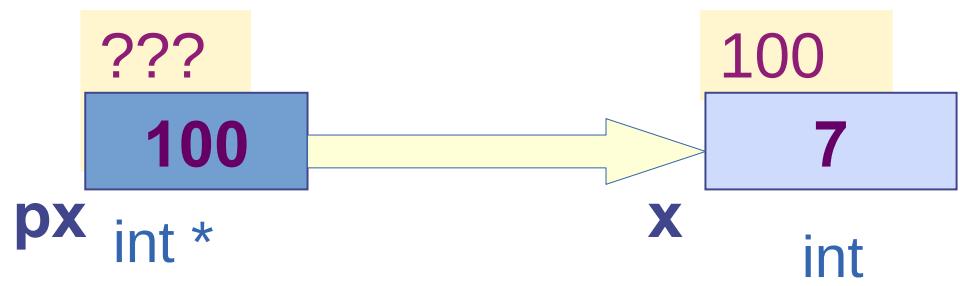


- `* px = 23;` // `*` - разыменование `x = 23;`
// записать **int** по адресу, который хранится в `px`
// потому что переменная `px` типа **int ***
`* px = * px + 4;` // `x = x + 4;`

адрес (pointer), значение (value)



```
printf ("%d", x);  
printf ("%p", &x);
```

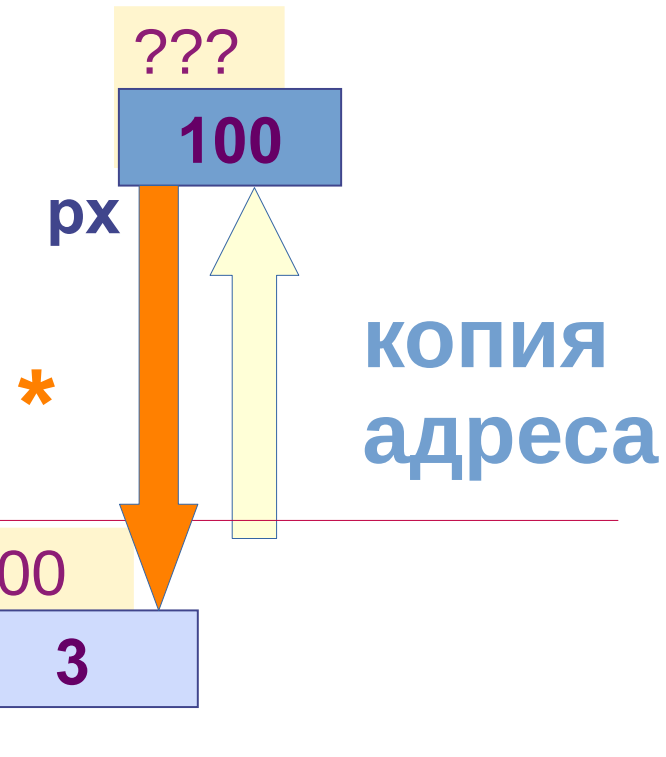


Передаем копию адреса

- Аргументы передаются только по значению

```
• void inc (int * px) {  
    *px = *px + 1;    // (*px)++;  
}
```

```
int main ( ) {  
    int a = 3;  
    inc (&a);  
    inc (&a);  
    printf("%d\n", a);    // ??  
}
```



Вернуть часы и минуты

- Возвращать 2 значения нельзя
- Можно передать адреса 2 переменных, по которым сохранить результаты.
- ```
void mm2hm (int mm, int * ph, int * pm) {
 // mm — минут с 00:00
 // ph, pm — указатели на часы и минуты
 * ph = ;
 * pm = ... ;
}

int main () {
 int h, m;
 mm2hm (135, &h, &m);
```

# Когда прибудет поезд?

- Отправление электрички в h1:m1.  
Прибытие электрички h2:m2.
- Сколько времени ехала электричка hh:mm?
- Электричка не ездит далеко. Она не может ехать больше 24 часов.
- Электричка может отправиться в 23:45 минут с Тимирязевской и быть в 00:07 минут на Долгопрудной. Время в пути 00:21.

| Input          | Output |
|----------------|--------|
| 23:45<br>00:07 | 00:21  |

# Глобальные переменные - использование

- `int x = 2;`

```
void incr () {
```

```
 x = x+3;
```

```
}
```

```
int main () {
```

```
 printf("%d\n", x); // 2
```

```
 incr();
```

```
 printf("%d\n", x); // 5
```

```
 incr();
```

```
 printf("%d\n", x); // 8
```

```
 return 0;
```

```
}
```

x

2

# Глобальные переменные - свойства

- Определены вне всяких блоков и функций
- Область видимости — от места декларации до конца файла
  - поэтому пишут в начале файла
  - область можно расширить
- Время жизни — время выполнения **программы**

- Начальное значение **0**

Статическая область памяти

- Инициализация только константами или неявная

```
int x; // x = 0
```

```
int y = 7;
```

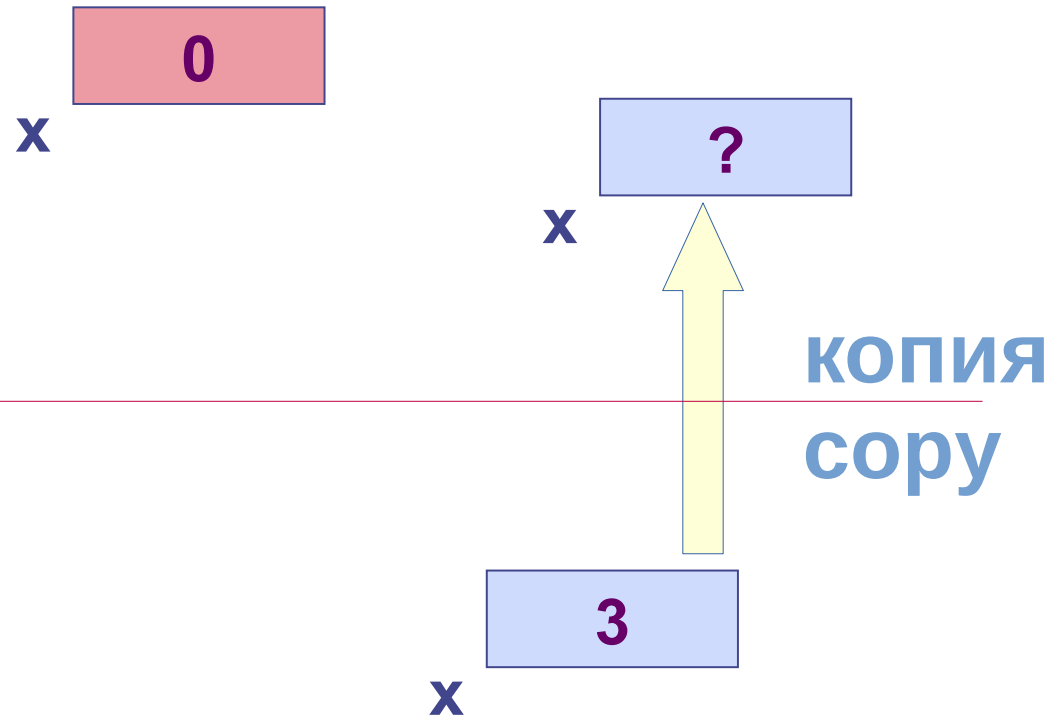
```
float z = sin(M_PI/4); // Ошибка
```

# Соккрытие переменных

- Глобальную переменную скрывает аргумент, аргумент скрывает локальная переменная

```
• int x;
void inc (int x) {
 x = x + 1;
}
```

```
int main () {
 int x = 3;
 inc (x);
 inc (x);
 printf("%d\n", x);
}
```



# Что будет выведено на печать?

- ```
void inc ( ) {  
    int x = 2;  
    x = x + 1;  
    printf("%d\n", x);  
}
```



```
int main ( ) {  
    inc ( );  
    inc ( );  
  
}
```

Модификатор `static`

- `static` `int x;`
- Область видимости
 - не изменяется (внутри функций)
 - ограничена одним файлом (вне функций)
- Время жизни — время выполнения программы
- Начальное значение 0
статическая область памяти.
- Инициализируется 1 раз.

Что будет выведено на печать?

- ```
void inc () {
 static int x;
 x = x + 1;
 printf("%d\n", x);
}
```

x 0

---

```
int main () {
 inc ();
 inc ();

}
```

# Что будет выведено на печать?

- ```
void inc ( ) {  
    static int x = 2;  
    x = x + 1;  
    printf("%d\n", x);  
}
```

x 

```
int main ( ) {  
    inc ( );  
    inc ( );  
  
}
```

Заключение

- Функции

```
float foo (float x, int n); // прототип
float foo (float x, int n) { // реализация
    float res;
    res = x * n;
    return res;
}

int main ( ) {
    float z;
    z = foo (3.5, -10);      // ВЫЗОВ
    return 0;
}
```